Deep Reinforcement Learning for Disturbance-Robust Ball-on-Tray Balancing

ME5418 – Machine Learning in Robotics | Stage 3: Learning Agent Code & Report

This stage turns the stage 2 networks into learning agents: complete training loops for both algorithms of the proposal, **PPO** and **SAC**, the proposal's **staged domain randomisation**, and the tools to evaluate a trained agent against the PD and LQR baselines and watch it in the 3D view. Everything is written against `torch.nn`, with no RL framework, and runs on one CPU core. The code is complete and runnable; first reference runs already learn: PPO reaches the LQR level in about 7 minutes. On the 100 test seeds PPO beats LQR on 77 of them but drops one hard start; under noise, lag and delay, PPO trained in those conditions beats LQR by 30 points. SAC learns faster per sample but levels off below PD.

1. Code Explanation

Stages 1–2 are unchanged apart from additions to `networks/losses.py` and a `--checkpoint` option in `demo.py`. Stage 3 adds the `agents/` package and three scripts.

File	Content
`agents/common.py`	Curriculum, parallel environments (`VecEnv`), evaluation on fixed seeds, CSV logging.
`agents/ppo.py`	PPO loop: rollouts, GAE, minibatch epochs, learning-rate annealing; optional asymmetric critic.
`agents/sac.py`	SAC loop: replay buffer, target critic (Polyak), automatic entropy tuning, random warm-up.
`networks/losses.py`	+ `gae` (GAE(λ) with bootstrapping at truncation), + `sac_alpha_loss` , + `use_min` option of the SAC actor loss.
`train.py`	Command line: `--algo ppo|sac` , `--curriculum` , `--realistic` , `--asymmetric` , `--quick` ; writes `runs/<name>/` .
`evaluate_agent.py`	Agents next to PD and LQR on the test seeds 0–99, with paired differences; `--realistic` .
`plot_training.py`	Learning curves with the PD and LQR levels (Fig. 1).
`tests/test_agents.py`	17 tests: GAE by hand, `VecEnv` , curriculum, replay buffer, Polyak, α direction, smoke runs, reproducibility.

1.1 Pipeline and shared pieces

`train.py` builds a configuration (saved as `config.json`) and calls `train_ppo` or `train_sac` . Both loops draw experience from `VecEnv` , update the stage 2 networks with the stage 2 losses, append a row to `progress.csv` , and every 50k (PPO) / 10k (SAC) steps evaluate the deterministic policy on seeds 500–519, saving `best.pt` and `last.pt` . **Curriculum.** The proposal's "fixed friction and no disturbances first, then progressively randomise" is four stages: fixed $c_{rr} = 0.02$, no pulses (10 % of steps) → $c_{rr} \in [0.005, 0.05]$ (15 %) → + pulses of 0.5–1.25 m s^{-2} (15 %) → full range, pulses up to 2.0 m s^{-2} (60 %); `--realistic` adds noise, lag and delay to the last stage. A stage changes at each environment's next episode, never inside one. **VecEnv** steps N environments and batches the network call. We wrote it instead of using `gym.vector` because both loops need the *true* last observation of an episode (and its `info` , for the asymmetric critic) separately from the observation after the automatic reset, and every episode needs its own seed for exact reproducibility. **Seeds.** Training episodes use seeds ≥ 100,000; model selection uses 500–519; the final comparison uses 0–99, the seeds of the stage 1 baseline table, seen by nothing else.

1.2 The two training loops

	PPO (`agents/ppo.py`)	SAC (`agents/sac.py`)
Data	on-policy: 8 envs × 256 steps = 2,048 per iteration, used once	off-policy: 1 env, every transition kept in a 300k replay buffer; 5,000 random warm-up actions
Update	10 epochs × minibatches of 256; clipped loss (ε = 0.2), value weight 0.5	1 update per step, batch 256: twin-Q critic, then actor, then α, then target
Value target	GAE, γ = 0.99, λ = 0.95	$r + \gamma(1-d)[\min Q'(s', a') - \alpha \log \pi(a'\|s')]$ from the **target** critic, τ = 0.005
Exploration	free log σ, no entropy bonus (σ decays 0.61 → 0.10)	α learned so that the entropy tracks −2 (= −action dim.)
Stabilisers	advantages normalised per minibatch; grad-norm clip 0.5; lr $3{\cdot}10^{-4}$ → 0 linearly	actor maximises min(Q1, Q2); critic frozen during the actor's backward pass

Both scale the reward by 0.1 for learning (value targets near 10 instead of 150; logs are unscaled), and both treat the two episode ends differently: a fall (`terminated`) is terminal, the 500-step limit (`truncated`) is not, so

$$\delta_t = r_t + \gamma\,(1 - term_t)\,V(s_{t+1}) - V(s_t), \quad A_t = \delta_t + \gamma\lambda\,(1 - done_t)\,A_{t+1}$$

with s_{t+1} the *true* last observation, not the next episode's first one that an auto-resetting environment returns. A balanced episode always ends by truncation, so treating it as terminal would teach the critic that balancing longer is worth nothing; a hand-computed GAE test pins down both cases.

1.3 First results

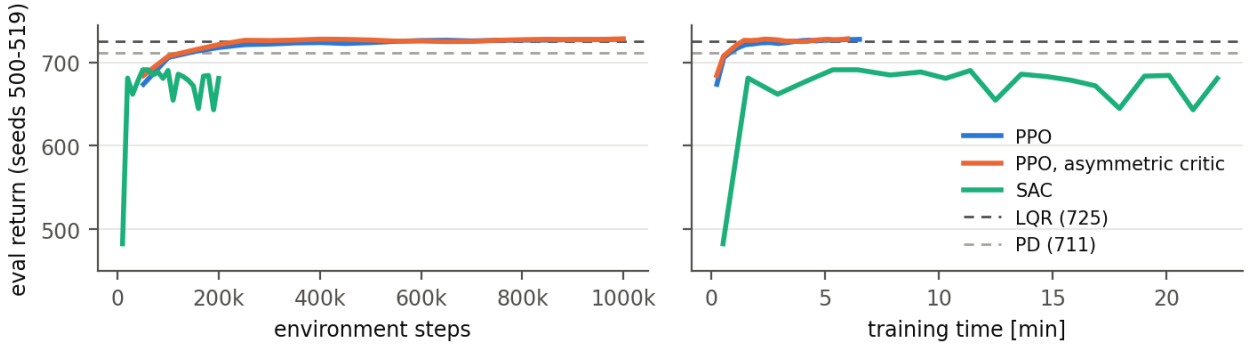


Fig. 1. Evaluation return (deterministic policy, seeds 500–519, full range) of the reference runs, against environment steps and training time; dashed: LQR and PD on the same seeds. One training seed per agent, one CPU core each: PPO 1M steps in 6.6 min, SAC 200k steps in 22 min.

Condition	Policy (one training seed each)	Return	Survival	Goal %	vs LQR, paired
Ideal	PD	712.6 ± 11.7	100 %	93.4	—
	LQR	724.4 ± 8.3	100 %	96.1	—
	PPO	720.0 ± 71.8	99 %	95.1	−4.4, better on 77/100
	PPO, asymmetric critic	720.6 ± 71.8	99 %	95.3	−3.8, better on 83/100
	SAC	692.0 ± 21.0	100 %	89.8	−32.4, better on 1/100
Noise, lag 0.05 s, 1-step delay	PD	658.5 ± 68.3	99 %	86.7	—
	LQR	627.4 ± 111.1	97 %	78.8	—
	PPO trained on the ideal task	386.7 ± 118.3	78 %	13.8	−240.7, better on 2/100
	PPO trained with `--realistic`	657.3 ± 69.8	99 %	81.1	+29.9, better on 77/100

Table 1. `evaluate_agent.py` on the test seeds 0–99 (full randomisation range; max. return 750). Each agent is its `best.pt`, chosen on seeds 500–519. "Paired" compares the same seed: same start, c_{rr} and pulse schedule.

2. Existing Codes / Libraries

What we used. PyTorch 1.13.1 remains the only ML dependency: `optim.Adam`, `nn.utils.clip_grad_norm_`, autograd, `torch.save`. gym 0.26.2 provides only the environment API; numpy runs the replay buffer and the episode bookkeeping; matplotlib draws Fig. 1. **What we wrote.** Both training loops, GAE, the replay buffer, the target update, entropy tuning, the curriculum, the vectorised environment and the evaluation are ours. As in stage 2 we did not use stable-baselines3 or RLlib: they would hide exactly what this stage is about, current stable-baselines3 requires `gymnasium`, and its PPO gives actor and critic the same input, so the asymmetric critic would need a custom policy class anyway. **What we borrowed.** Hyperparameters, not code: the PPO values are close to the MuJoCo settings of Schulman et al. (2017) and the stable-baselines3 defaults; the SAC values and the target entropy −dim(A) follow Haarnoja et al. (2018b). **What we had to do to make it work.** `torch.set_num_threads(1)` (fastest for 22k-parameter networks and bit-reproducible: two runs with one seed give identical weights, which a test checks); every checkpoint stores its own configuration so that `evaluate_agent.py` and `demo.py` can rebuild any run; the code stays Python 3.7-compatible.

3. Reflections and Lessons Learned

3.1 Changes from the proposal and from stage 2

- **The curriculum became concrete.** The proposal said "progressively"; we had to choose stages and fractions (§1.1). The first stage turned out to be sufficient for a surprisingly good policy: PPO evaluated on the *full* task after 51k steps of only fixed- friction, pulse-free training already scores 674 with 100 % survival on 20 evaluation seeds: it generalises to frictions and pushes it has not seen yet.
- **SAC's actor now uses min(Q1, Q2)** instead of stage 2's Q1, and α is learned instead of fixed at 0.2 (stage 2 deferred both). In the reference run α fell from 0.2 to below 0.003 within 30k steps while the policy entropy settled at the target −2.
- **No PPO entropy bonus** (stage 2 default: 0.01). With a state-independent log σ, exploration is set by σ itself, which PPO lowers as the policy improves (0.61 → 0.10 over the run); we have not yet tested whether a bonus helps.
- **Reward scaling** (×0.1, learning only) was not in the proposal. It keeps value targets near 10 instead of 150, so the value loss stays on the scale of the clipped policy loss it is added to; we chose it by reasoning, not by an ablation.

3.2 Lessons

- **Mean return hides the tail.** On the test seeds PPO beats LQR on 77/100 seeds (+2.4 on the other 99), yet its mean is lower, because it drops the ball once: seed 0 starts near a corner with a 1.41 m s^{-2} push from t = 0.03 s, a rare combination. LQR saves it. The proposal's success-rate metric is the right primary metric; the final version needs targeted hard-start tests.
- **Train in the conditions you deploy in.** Under noise, lag and delay, the PPO policy trained on the ideal task collapses (386.7, 78 % survival), plausibly because the precise, high-gain behaviour it learned is what lag destabilises. The same run with the realism parameters in its last stage reaches 657.3 (99 %): 29.9 above LQR, which degrades for the same reason, and level with PD. Its learning curve shows the effect directly: while it trained on the ideal early stages, its realistic evaluation score fell from 643 to 251, and it recovered only once the realistic stage began.
- **Sample efficiency ≠ speed.** SAC scored 681 after 20k steps, where PPO had 674 after 51k, but each SAC step costs a gradient update: its 200k steps took 22 min, PPO's 1M took 6.6. SAC then levelled off at 680–690, below PD. We do not know why yet; candidates are the target entropy, the reward scale, and simply too few steps.

3.3 Remaining changes towards the final version

Several training seeds per configuration with confidence intervals; the proposal's two remaining baselines (LQR at nominal friction, and the grid-searched fixed gain); held-out conditions including 3 m s^{-2} pulses; the asymmetric critic and `--curriculum full` as ablations (the asymmetric critic reached each level slightly earlier in our single run, 726 vs 721 at 252k steps, and ended equal: one seed cannot answer stage 2's open question); hard-start episodes to close the tail failure; and a diagnosis of SAC's plateau.

4. How to Run

```
python train.py --algo ppo --quick                    # ~15 s, every code path (also --algo sac)
python train.py --algo ppo|sac [--realistic] [--asymmetric]   # PPO 1M steps ~7 min, SAC 200k steps ~22 min
python evaluate_agent.py trained/*.pt [--realistic]            # the four reference agents vs PD, LQR (Table 1)
python demo.py --checkpoint trained/ppo_staged_seed0.pt --policy lqr+agent    # 3D view
```

All 65 tests (14 environment, 7 renderer, 8 realism, 19 network, 17 agent) pass; `README.md` documents every option.